Assignment #2: Federated Learning and AI Model Serving

Course:	CS 595–003 (“Decentralized ML Systems”)
Instructor:	Dr. Nathaniel Hudson
Instructor Email:	nhudson5@illinoistech.edu
TA:	Aravind Balaji Srinivasan
TA Email:	asrinivasan@hawk.illinoistech.edu
Deadline:	October 6th, 2025 at 11:59pm with 1-hour grace period (Canvas deadline is 1:00am CT, October 7th, 2025)

1 Description

For this project, you will implement two programs to implement: *(i)* a **Federated Learning (FL)** system to train a global model, and *(ii)* **a web application** that processes inference requests using your global model from the FL simulation—deployed on the Chameleon Academic Cloud. Your web application will be containerized with Docker for portability.

The neural architecture you train should be implemented in PyTorch, but you are free to use other Python tools to reduce boilerplate code (e.g., PyTorch Lightning, PyTorch-Ignite, Skorch). However, **use of FL frameworks (e.g., Flower, Flight, APPFL) is strictly prohibited**.

Also, you will be required to submit a **2-page written report** in the style of an academic paper. This report will provide a high-level description of your project, its aims, motivations, and central questions. The report will also need to include key results from training and evaluating your neural network models. More information on the specific requirements of this report are provided below in Section 3.3.

> **NOTE:** As per the course syllabus, late submissions will not be accepted and will receive a 0.
>
> Additionally, if the size of your `.zip` for your submission exceeds 10MB in size, your final grade will be docked by 3 points.

2 Deliverables

To complete this assignment, you must submit one `.zip` file on Canvas that contains the following items:

1. A folder containing the following all the **source code** for your submission with instructions for how to run your model training script. Additionally, your folder with source code should also contain:
   (a) A `` `requirements.txt` `` file that has the libraries that you used to run your code. This can be generated automatically by running `` `python -m pip freeze > requirements.txt` `` in your Python environment.
   (b) A `Dockerfile` that describes how to build your Docker web application.
   (c) A `` `README` `` file that contains all instructions to setup your environment for your code, namely run your FL simulation and web application. The Python version number should also be included (which is given by `` `python --version` ``).
   More specifically, this file **must provide instructions for launching your web application on a Chameleon node such that it can be used from any Internet-connected device not directly on the local Chameleon network**.

2. Data files that contain the results of your training (e.g., data for loss curves); this can be stored as a `.csv` or any other file supported by Pandas (e.g., `.json`, `.xls`, `.parquet`).

   Do NOT include the training dataset itself in your `.zip` file.

   Your data file should have *at least* the following columns (you may name them differently):

time	round	batch_num	client_id	train_loss	train_acc	...
...	...	...	...	...	...	...

3. A `.pdf` document containing a 2-page report of your results, methodology, and takeaways.

   It is expected that this report is written like a scientific research paper. As such, this must be written in LaTeX using the IEEE template.

   A template on Overleaf to get started can be found here. I recommend cloning this template in Overleaf to start your report.

3 Requirements

For this project, in addition to your written report (described in §3.3), you will write **two** Python programs:

1. a **federated learning simulation** for image classification with at least 64 (simulated) participating devices which will save the final global model maintained by the (simulated) coordinating server, and
2. a **web application** which will have a simple user interface that allows users to upload images for inference with the final global model trained from the FL simulation.

Your code must be written in Python 3. More details about the requirements of these two programs are presented below in §3.1 and §3.2.

3.1 Code: Federated Learning Simulation

For this assignment, your FL simulation implementation **must** incorporate multi-threading or multi-processing via the `ThreadPoolExecutor` or `ProcessPoolExecutor` classes (respectively).

Note, at the end of your FL simulation (ideally when your model has converged in predictive performance), you will need to **save the final global model**.

The structure of your simulation can be largely based on the pseudocode of the `FedAvg` algorithm by McMahan et al. (2017). You will use the `Executor` to do the local training (or `LocalUpdate` in the pseudocode) parts in parallel or concurrently.

3.1.1 Executor Requirements

Below are the requirements (and tips) related to how the `Executor` is used for your FL simulation.

- Your FL simulation must be made up of *no fewer than* 16 simulated devices (i.e., devices with "private" data partitions).
- When implementing your FL simulation, you must have a **local training job** function that does local training for a simulated device with its private data partition. This will be a function that you pass into `Executor.submit(job, ...)` to executor.

- **Tip:** In the initialization of your `Executor`, you can set the `max_workers` argument to control the number of devices that will train concurrently, or in parallel.
- **Tip:** Be mindful about how you set `max_workers`. For `ProcessPoolExecutor`, you may run into bad system utilization because processes are so heavy relative to threads.
- **Tip:** Additionally, if you choose to use PyTorch Lightning, make sure that you initialize the `Trainer` within the **local training job** function that you submit to the `Executor`. Instances of the `Trainer` object maintain some state about training and you do not want to mix that state with the training done for other simulated devices.

3.1.2 Neural Network Requirements.

For simplicity, you will not be required to design your own neural network architecture. In many real-world scenarios, you will take advantage of model architectures that have been shown to work well.

So, for this assignment, you will use one of the widely-available ImageNet model architectures which are provided by the `torchvision.models` module. A list of the neural networks available in this module can be found here. These models are trained on the ImageNet dataset which is a much larger vision dataset than CIFAR-10/100. The ImageNet dataset has 1000 labels. So, the model architectures will need to be adapted.

An example of how you can load and adapt one of these architectures is given below:

```
from torchvision import models

alexnet = models.alexnet(weights=None)

# Adapt AlexNet to support 32*32 RGB images.
alexnet.features[0] = nn.Conv2d(
    in_channels=3,
    out_channels=64,
    kernel_size=3,
    stride=1,
    padding=1,
)

# Adapt the final Linear layer for `num_classes`.
alexnet.classifier[-1] = nn.Linear(
    in_features=4096,
    out_features=num_classes,
)
```

3.1.3 Data Requirements

For this project, for simplicity you will once again use either the CIFAR-10 or the CIFAR-100 dataset.

- the **train partition** will be used for the local training partitions on each of your simulated worker devices in your FL simulation, and
- the **test partition** will be used for testing the global model after aggregation towards the end of each communication round.

You will need to design your own method for splitting up the original train partition across all your workers in your FL simulation. This will need to be done in a way where you emulate non-iid data distributions.

A simple, naïve example of how to split up the original dataset is given:

```
import typing as t

ClientID: t.TypeAlias = int

```

```python
from torch.utils.data import Subset
from torchvision import datasets

cifar10 = datasets.CIFAR10(..., train=True)

client_data: dict[ClientID, Subset] = {
    0: Subset(cifar10, indices=list(range(0, 10))),
    1: Subset(cifar10, indices=list(range(10, 100))),
    ...
}
```

The `Subset` class (from above) allows you to "subset" a given (mappable) dataset to only include the given indices. It's a simple abstraction and is recommended for designing your data partitioning method. However, the challenge will be how you assign the indices to ensure non-overlapping datasets and non-IID data distributions.

For this task, it is recommended you take advantage of the probability distribution methods in `numpy`/`scipy`.

You will be required to include a visualization of your data distribution.

3.2 Code: Web Application

Once your global model is trained to convergence from the FL simulation, you will then "deploy" it as a web application on the Chameleon Academic Cloud. Your application will be containerized. Further, it should be developed in a way where you can host it on the Chameleon cloud and submit requests (and receive responses) over the Internet from another device.

3.2.1 Web Framework

You can use whichever web application you would like. So long as you are able to run the server for the web application on a Chameleon node and configure it to be accessed remotely.

You are not required to design a web UI with HTML. You are free to if you wish. However, a simple REST API is sufficient. When you submit your `README` file, just clearly describe how to:

1. run your server
2. submit a request for inference

Below are some recommended frameworks for building your web application:

1. Ray Serve (recommended w/FastAPI)
2. FastAPI (recommended w/Ray Serve)
3. Flask
4. Django

3.2.2 Docker

Docker is required for this assignment to containerize your web application, ensuring it can be reliably deployed on the Chameleon Academic Cloud and accessed remotely. Containerization provides several key benefits:

To have your application accessible from outside Chameleon, you will need to setup your lease on Chameleon to use a Floating IP Address and then run the application with address `0.0.0.0`.

A recommended, easy-to-use image on Chameleon is `CC-Ubuntu16.04`. Installing Docker on this image can be done via these instructions.

To meet the assignment requirements, follow these steps:

1. **Create a Dockerfile:** This file defines how to build your Docker image. It should:
    - Set up the Python environment.
    - Install required packages from your `requirements.txt`.
    - Copy your trained `model.pt` file and web app code into the container.
    - Define how to run your web server (e.g., using FastAPI or Flask).
2. **Build and Publish the Docker Image (Optional):**
    - Use `docker build` to create the image.
    - Push it to a container registry (e.g., Docker Hub or GitHub Container Registry).
3. **Deploy on Chameleon Cloud:**
    - Pull the image to your Chameleon node; if you did not build and publish Docker image, then build one from the `Dockerfile` on your Chameleon node.
    - Run the container using `docker run`, exposing the necessary ports.
    - Ensure the server listens on `0.0.0.0` so it can be accessed from outside the Chameleon network.
4. **Document Everything in Your README:**
    - Include instructions for building and running the Docker container.
    - Specify the Python version and how to access the web app remotely.

More information on **exposing ports** in Docker can be found here.

3.2.3 Additional Requirements

- You **must** implement your code in standard PyTorch (i.e., no Lightning, PyTorch-Ignite, Skorch, Keras). Any use of supplemental frameworks for designing, implementing, and evaluating the model will result in a 0 towards the "Meets Requirements" portion of your grade on this assignment (see Table 1).
- You *will* be graded on **code style**. Please refer to the Code Style Guide—available on the course Canvas page for more information.
- Part of the challenge of this project is finding a good training configuration (e.g., optimization algorithm, learning rate, batch size, regularization method if needed).
- The plot showing your learning curve from training must show the training loss and/or training accuracy as well as validation loss and/or validation accuracy.

3.2.4 Tips

- **START EARLY!** There are a lot of things to cover to successfully complete this assignment.
- **Develop locally** to setup your web application on top of Docker. Then, when you have a good feel for it, you should be able to port your Docker app much more easily to Chameleon.
- Note, you will need to install Docker to your Chameleon node because they are `baremetal` nodes. So, be sure to give yourself enough time when you create your lease on Chameleon.
- If you have access to a GPU, you should try to take advantage of GPU acceleration. In PyTorch, this is easy. For tensors (e.g., data samples) and models, you just need to explicitly transfer them to the GPU before you do calculations (e.g., forward and backward passes). When you record your results (e.g., loss for loss tracking), you will need to transfer those values to the CPU first.

```python
import torch.nn as nn
module = nn.Linear(1, 1)
data = torch.tensor([10.])

# Note: You have to reassign data variables, but not modules.
module.to("cuda")  # 'cuda' is the GPU acceleration backend for NVIDIA GPUs
data = data.to("cuda")
model(data)
# ...
```

- If you have an Apple Silicon macOS device, you can use your Mac's GPU to accelerate local model training. You just have to use the `'mps'` backend instead of `'cuda'`.
- Debug your code on small subsets of the data to work on small errors and finalize how you get results from your training loop to produce your plots. Your "big runs" should be done towards the end of your development timeline. You can create smaller subsets of data using the `Subset` class from `torch.utils.data` module.
- It is recommended you use `matplotlib` and `seaborn` for plotting your results. Additionally, it is recommended that you save your results as `.pdf` files; LaTeX lets you embed `.pdf` files directly.
- For saving your results, it is recommended that you record your results using a list of records. A "record" is a list of `dicts` (i.e., `list[dict[str, typing.Any]]`). A single dictionary here will record a single row of data. You can convert this into a tabular dataset using `pandas.DataFrame.from_records()`.

3.3 Written Report

Your report must be written in LaTeX using the conference paper template provided by IEEE. A template in the IEEE format is available on Overleaf to get started can be found here; it is recommended you clone this template in Overleaf to start your report.

This document is expected to be written similar to an academic research paper, it should have clear sections. Following the title, author name, and abstract, it is encouraged that your report's sections follow a structure like:

1. **Introduction:**

   This section should introduce the problem (i.e., image classification with non-IID data) and motivate why federated learning and model deployment is worthwhile.

2. **Methodology:**

   Here, you should have two subsections.

   (a) The first one will be focused on your FL simulation and should discuss in detail how you implement FL, both the training loop and the data distribution method.

   (b) The second one will be focused on your web application. It should focus on web framework and other technologies used to implement your web application.

3. **Experiments & Results:**

   Describe the application you try to develop and justify the design decisions you made for developing and deploying it.

4. **Conclusion & Future Directions:**

   Conclude the report by briefly discussing the motivation, the problem, and how your results answer the core question this report is focused on. Also, provide some ideas for future questions you might extend this work if you had to continue this project.

Category	Requirement	Points
Code: FL Simulation	Correctness	2.5
	Non-IID data distributions	0.5
	Correct use of `Executors`	0.5
	Style	0.5
Code: Web Application	Works correctly	1.5
	Accessible from remote devices	1.0
	Containerized	1.0
	Style	0.5
Written Report	Completeness	1
	Clarity	0.5
	Data Included	0.5

Table 1: The grading rubric that breaks down how your grade will approximately be calculated. Remember, this assignment makes up 10% of your total grade (i.e., each point is 1% of your grade). The instructor and TA maintains the right to adjust this as needed. Note: The "Data Included" portion of the written report part of the rubric relates to the data files with your results that support your figures.

NOTE: This report is meant to be *similar* to an academic research paper. It is not expected to have all of the pieces of a standard paper (e.g., related work, background, large number of references). Your report should be succinct and self-contained.

3.3.1 Additional Requirements:

- Reports must be written by the individual student.
- Large Language Models (LLMs) should not be used in the writing of this document.
- **Your report must include at least THREE results as figures with discussion**:
  1. A **learning curve** that, on the x-axis, shows the average **training accuracy/loss** from your simulated worker devices alongside the **testing accuracy/loss** that is done with the global model after aggregation towards the end of each round. The y-axis should be communication rounds.
  2. A plot that clearly shows the data distribution across all your simulated worker clients. This should clearly show that your data distribution is non-IID.
  3. A screenshot of your web application in action with a successful inference request.

4 Frequently Asked Questions

4.1 How should I ask for help?

Preferably on the Federated and Decentralized Learning Module Discussion on Canvas.

You are encouraged to discuss strategies and share tips/tricks with one another on the topic discussion page. **However, you cannot share large blocks of code**.

- For clarification, it is okay to provide **small code snippets** (i.e., 1–3 lines of code) that highlight a helpful function you can import within PyTorch, Pandas, etc.

You are free to ask questions of the instructor or the TA via email and/or office hours. However, it is strongly recommended that you also ask on the course Canvas page. It is likely that many students will have the same questions and it would benefit everyone to keep discussions more centralized.

4.2 Can I use PyTorch Lightning or other similar frameworks?

Yes!

For this assignment, you are free to use PyTorch Lightning or other similar libraries (e.g., PyTorch-Ignite, Skorch) for reducing the amount of boilerplate needed to train a model with PyTorch.

4.3 Can I use Federated Learning frameworks?

No.

You must write your own code for simulating federated learning. Use of frameworks like *Flower*, *Flight*, *APPFL*, and *TensorFlow Federated* are **strictly prohibited** and will result in a 0 grade for the FL simulation portion of your grade.

4.4 Am I required to Chameleon Cloud?

Yes.

Chameleon Cloud will be used for demonstrating that your application can be reached from remote devices.

4.5 Can I use Large Language Models (e.g., ChatGPT) for writing the report?

Yes, but *only* for simple revisions (e.g., grammar, polishing English).

However, I do not recommend it. Part of research is learning how to communicate your ideas in a way that others can learn from your insight. Writing is also a good process to better understand *your* own thoughts. If you offload the task to a Large Language Model, you are only robbing yourself of that opportunity.

However, if you choose to use an LLM to aid you in written your report, please provide a transcript of your prompts at the end of your report as an *Acknowledgments* section. This will look like this in your paper:

```
...
\section{Conclusions \& Future Directions}
...

%% Note, the \section* here. This makes this section unnumbered.
\section*{Acknowledgment}
Below is a text record of the prompts I gave to the XXXXX:
...

\bibliographystyle{ieeetr} %% References go here.
\bibliography{refs}
```